

Homework 2 | Deep Learning | ELE2346

Marcelle Alexandra Pereira da Silveira Rosa

Introdução

Diferente das redes feedforward tradicionais, as redes neurais recorrentes (RNN) tem a capacidade de ‘memorizar’ informações de um passado mais distante, ou estados anteriores, sendo útil para implementação em tarefas que dependam de contexto temporal.

As primeiras RNNs enfrentaram problemas como a perda do gradiente ou sua explosão, inviabilizando a aprendizagem do modelo em sequencias longas. Para solucionar esses desafios, foram propostas variações mais sofisticadas como a Long Short-Term Memory (LSTM), uma arquitetura que utiliza portas de entrada, esquecimento e saída, além de uma célula interna de memória. Essas portas tem como função controlar o fluxo das informações, conforme relevância para aprendizagem do modelo.

Há também a Convolutional LSTM (ConvLSTM), essa arquitetura aplica convoluções internas da célula. Dessa forma, não perde a estrutura espacial dos dados e preserva a dimensão espacial durante o processamento temporal. Une as dimensões de espaço-tempo para reconhecer padrões.

Gated Recurrent Unit (GRU) é a versão simplificada da LSTM, combinando portas de entrada e esquecimento em uma única porta ‘*update*’. É mais leve computacionalmente, uma vez que possui menos parâmetros, mantendo um bom desempenho em determinadas aplicações, como aquelas que não precisam de uma memória de longo prazo tão extensa.

Por último, citamos DeepLSTM, uma arquitetura que ‘empilha’ camadas LSTM, formando uma rede mais profunda de aprendizagem. A saída de cada camada LSTM serve de entrada para a próxima, com o objetivo de extrair representações mais complexas, sendo útil quando há cenários com padrões hierárquicos, como reconhecimento de fala.

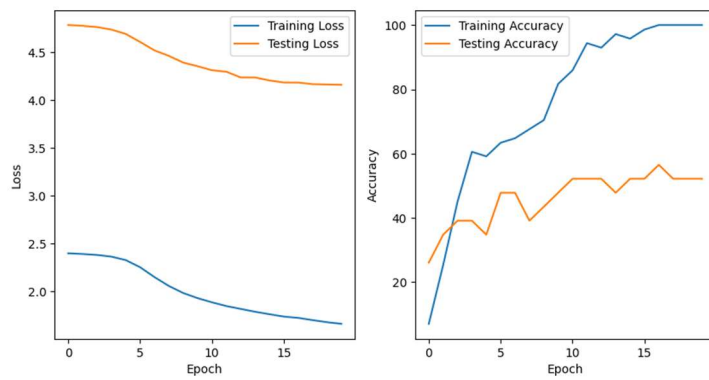
Treinamento e Resultados

O treinamento se deu a partir do *UCF Youtube Action Recognition dataset*, que classifica em 11 classes, diferentes esportes a partir de vídeos. Foram testadas diferentes arquiteturas, com diferentes valores para o parâmetro de *hidden_size*, que define a dimensão do vetor e a capacidade de memória e representação. Quanto maior o *hidden_size*, maior deve ser a capacidade do modelo em representar padrões complexos e armazenar informações.

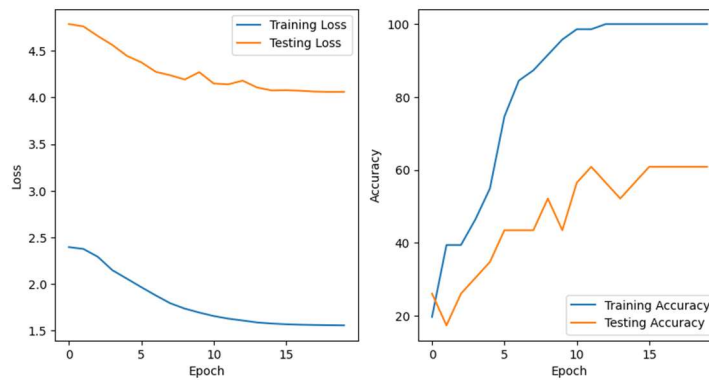
Abaixo é possível observar o resultado dos testes:

	Type	hidden_size	Parameter count	Training time (s)	Overall Accuracy	Avg. F1-Score
1	LSTM	64	0.18M	3.83s	46.99%	44.92%
2	LSTM	128	0.46M	4.80s	59.64%	58.17%
3	LSTM	256	1.32M	5.21s	62.35%	60.94%
4	LSTM	512	4.21M	7.03s	61.75%	60.58%
5	GRU	64	0.30M	5.17s	60.24%	55.21%
6	GRU	128	0.79M	6.63s	59.04%	56.39%
7	GRU	256	2.37M	7.32s	63.25%	59.55%
8	GRU	512	7.89M	10.17s	66.27%	65.37%
9	Deep LSTM (num_layers=2)	64	0.56M	6.07s	50.60%	47.16%
10	Deep LSTM (num_layers=2)	128	1.72M	7.68s	58.43%	55.93%
11	Deep LSTM (num_layers=2)	256	5.79M	10.19s	55.42%	52.04%
12	Deep LSTM (num_layers=2)	512	10.51M	13.2s	61.45%	59.80%
13	ConvLSTM	64	1.62M	35.79s	58.13%	53.73%
14	ConvLSTM	128	4.13M	39.04s	63.55%	62.12%
15	ConvLSTM	256	11.80M	64.23s	59.94%	57.81%
16	ConvLSTM	512	37.76M	175.63s	44.58%	35.92%

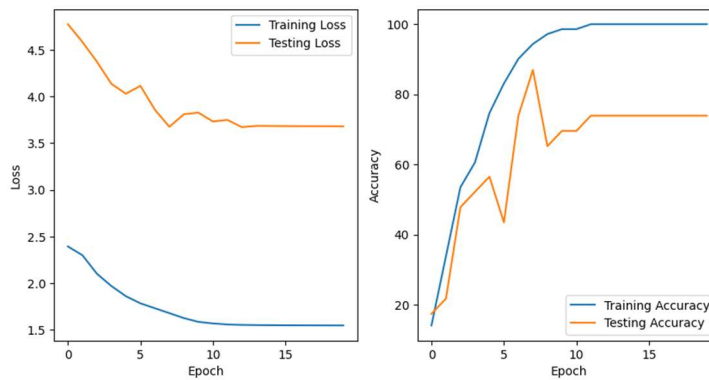
1.



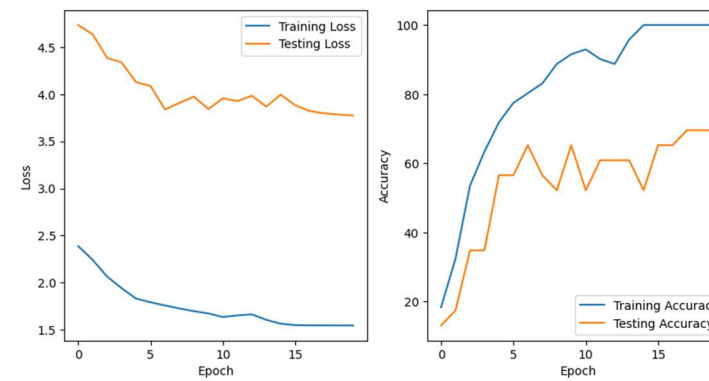
2.

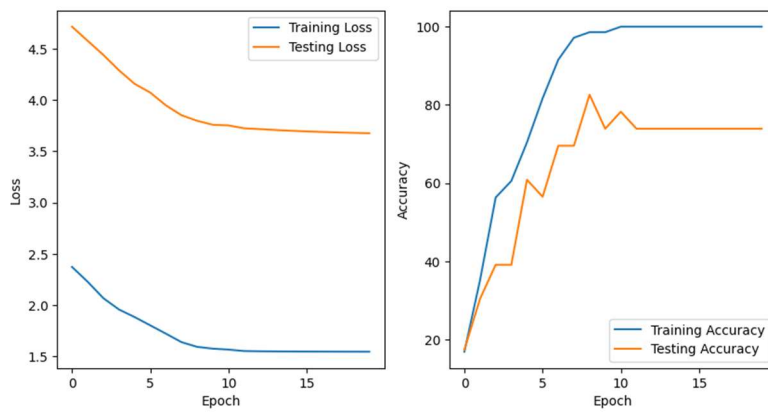
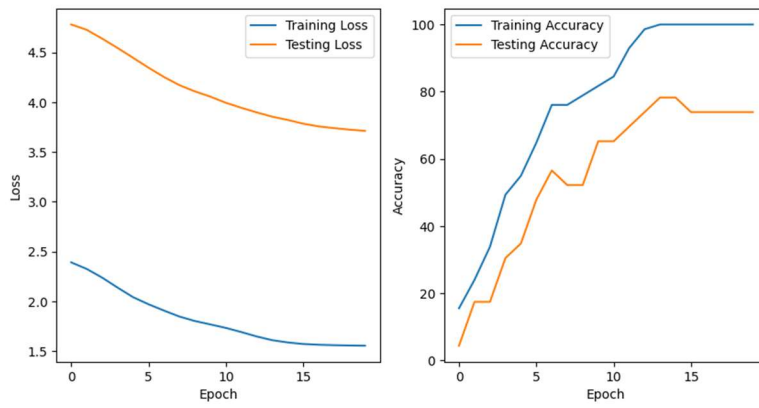
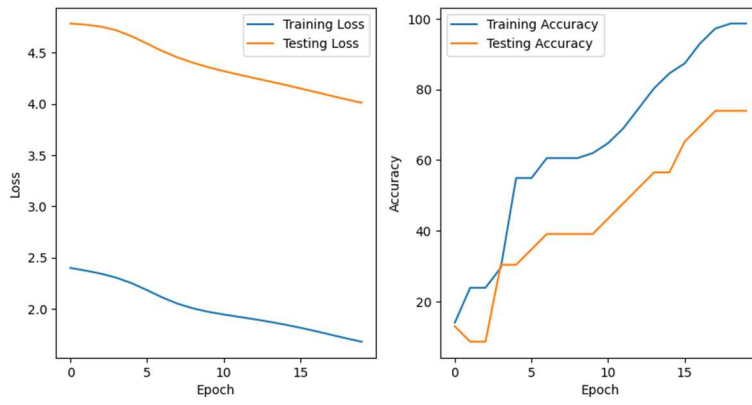


3.

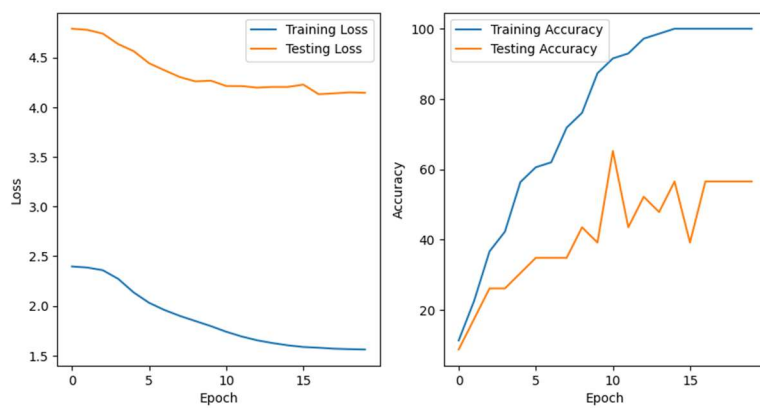


4.

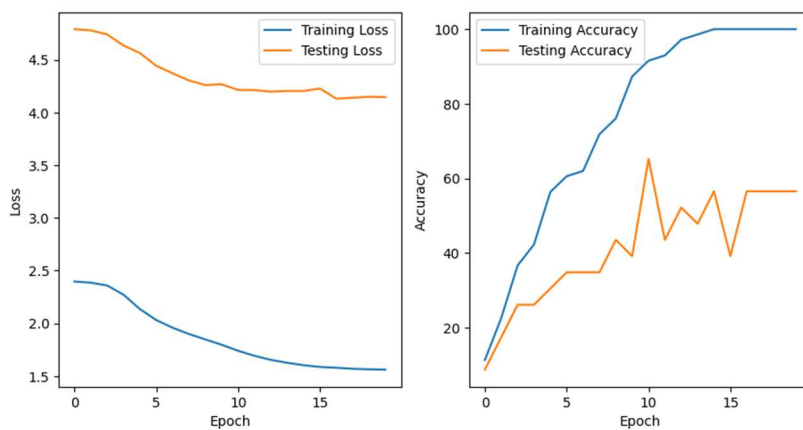




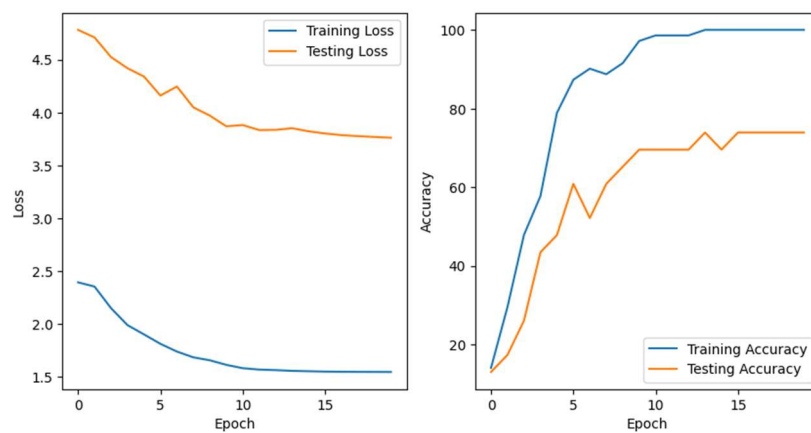
8.



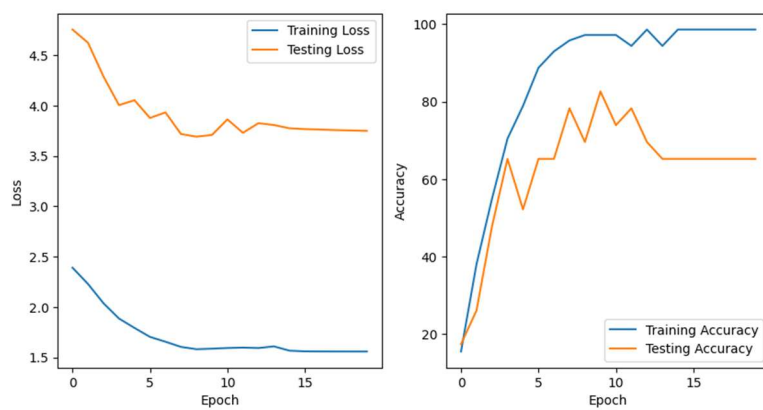
9.

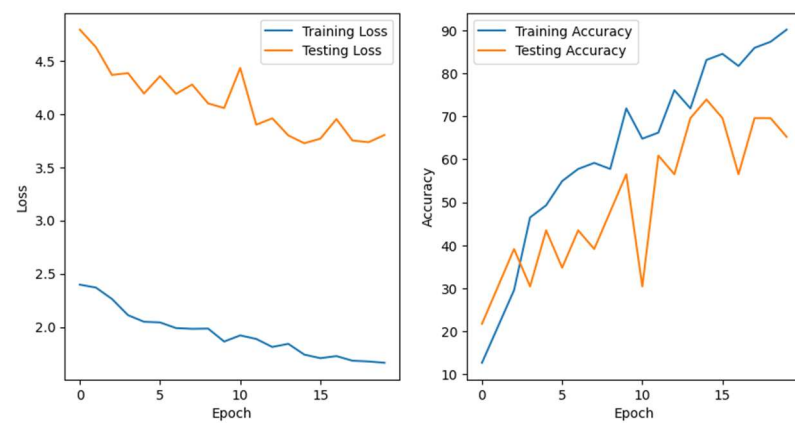
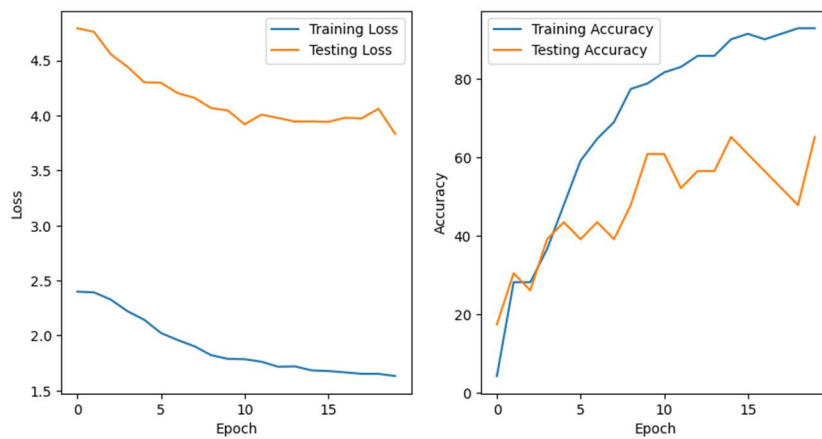
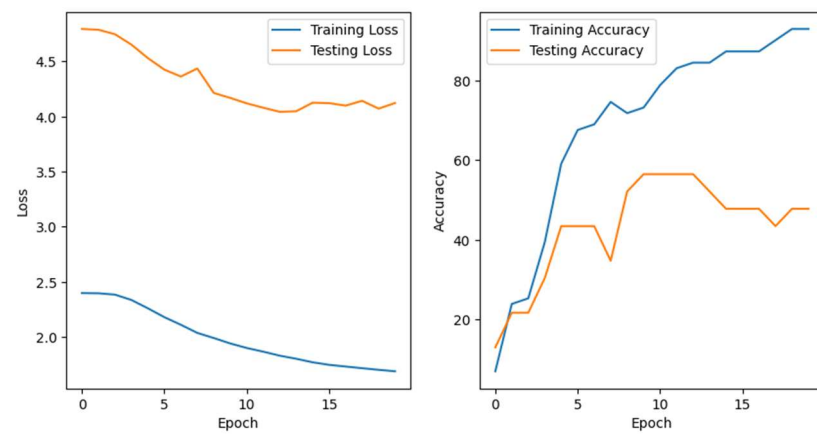
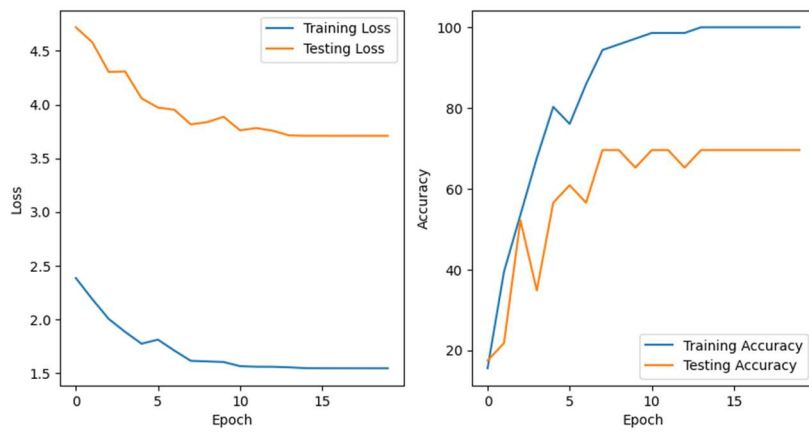


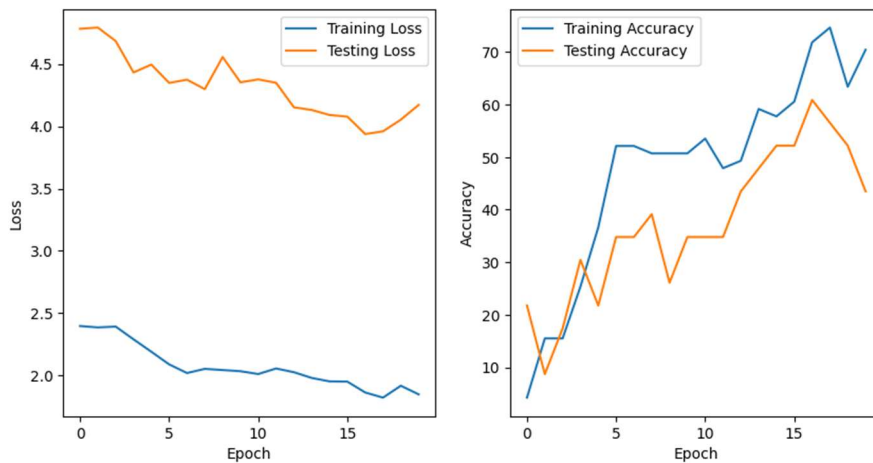
10.



11.







16.

Conclusão

Analisando os resultados, observa-se que, à medida que o `hidden_size` aumenta, as métricas de acurácia e F1-score tendem a melhorar, apresentando estabilidade para valores entre 256 e 512. A escolha de `hidden_size` = 256 apresenta o melhor custo-benefício, uma vez que a diferença entre os resultados é pequena e o custo computacional ao utilizar `hidden_size` = 512 é maior.

Os resultados obtidos utilizando a arquitetura GRU mostram como essa arquitetura cumpre o que propõe ao ser mais leve e manter um bom nível de acurácia quando comparada a outras arquiteturas, principalmente ao aumentar o número de parâmetros, utilizando `hidden_size` = 512.

Além disso, foi possível observar como uma rede mais profunda, como a DeepLSTM, nem sempre traz resultados melhores apesar de ter uma quantidade maior de parâmetros. Isso porque, o modelo pode se tornar tão complexo que acaba atrapalhando a aprendizagem.

Por fim, a convLSTM apresentou um treinamento mais pesado computacionalmente e não obteve resultados satisfatórios que justificassem a sua utilização.

Dessa forma, os experimentos a partir do *UCF Youtube Action Recognition dataset* indicam que modelos mais simples e equilibrados em termos de parâmetros,

podem oferecer melhor combinação entre desempenho e eficiência computacional, servindo como uma escolha mais prática para aplicações.

Notebook

[https://colab.research.google.com/drive/1crfF9NIMmxeiP0HB70QnLERPjAspPnH-
?usp=sharing](https://colab.research.google.com/drive/1crfF9NIMmxeiP0HB70QnLERPjAspPnH-?usp=sharing)